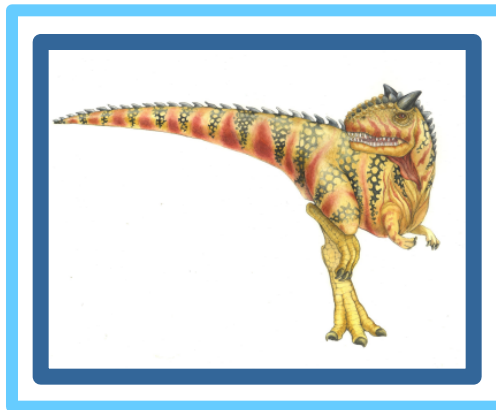


Chapter 7: Synchronization Examples





Classical Problems of Synchronization

- ❑ Classical problems used to test newly-proposed synchronization schemes
 - ❑ Bounded-Buffer Problem
 - ❑ Readers and Writers Problem
 - ❑ Dining-Philosophers Problem





Bounded-Buffer Problem

- ❑ Producer consumer problem
- ❑ Can we solve this problem using semaphore?
 - ❑ n slots in a buffer, each can hold one item
 - ❑ Two processes — producer and consumer
 - ❑ Producer insert data into the buffer and consumer removes the data from the buffer
 - ❑ Three Problems:
 - ❑ the producer must not insert data when the buffer is full.
 - ❑ The consumer should not try to remove data when buffer is empty.
 - ❑ Producer and consumer can't insert and remove data at the same time.





Bounded-Buffer Problem

[?] Solution:

- [?] Semaphore **mutex** — a binary semaphore used to acquire and release the lock. initialized to the value 1
- [?] Semaphore **full** — counting semaphore with initial value 0
- [?] Semaphore **empty** — counting semaphore and initialized to the value n (number of slots) - initially all slots are empty





Bounded Buffer Problem (Cont.)

 The structure of the producer process

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty); //wait until empty > 0 and then decrement  
empty  
    wait(mutex); //acquire lock  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex); // release lock  
    signal(full); //increment full variable  
} while (true);
```





Bounded Buffer Problem (Cont.)

 The structure of the consumer process

```
Do {  
    wait(full); //wait until full>0 then decrement full  
    wait(mutex); //acquire lock  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex); //release lock  
    signal(empty); //increment empty  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true);
```





Readers-Writers Problem

- ❑ A data set is shared among a number of concurrent processes
 - ❑ Readers – only read the data set; they do **not** perform any updates
 - ❑ Writers – can both read and write
- ❑ Problem – allow multiple readers to read at the same time
 - ❑ Only one single writer can access the shared data at the same time
- ❑ Several variations of how readers and writers are considered – all involve some form of priorities
- ❑ Solution – two semaphores and an integer variable
 - ❑ Semaphore **wrt_mutex** initialized to 1 – common to both reader and writer
 - ❑ Semaphore **mutex** initialized to 1 – ensure mutual exclusion
 - ❑ Integer **read_count** initialized to 0 – integer variable to keep track of the count of the processes that are reading the data





Readers-Writers Problem (Cont.)

 The structure of a writer process

```
do {  
    wait(wrt_mutex);  
    ...  
    /* writing is performed */  
    ...  
    signal(wrt_mutex);  
} while (true);
```





Readers-Writers Problem (Cont.)

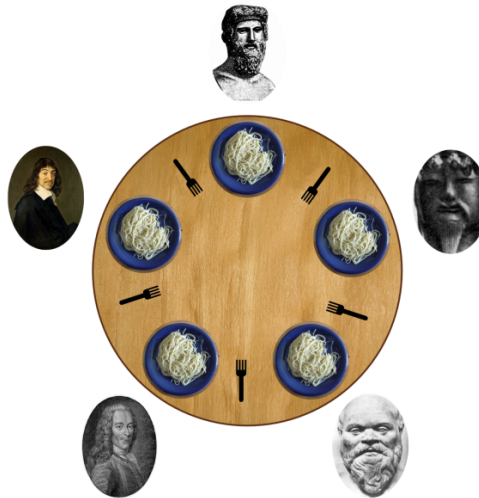
? The structure of a reader process

```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(wrt_mutex);  
    signal(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(wrt_mutex);  
    signal(mutex);  
} while (true);
```





Dining-Philosophers Problem



<https://en.wikipedia.org/>

- ❑ Philosophers spend their lives alternating thinking and eating
- ❑ Don't interact with their neighbours, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - ❑ Need both to eat, then release both when done
- ❑ In the case of 5 philosophers
 - ❑ Shared data
 - ▶ Bowl of rice (data set)
 - ▶ Semaphore **chopstick** [5] initialized to 1





Dining-Philosophers Problem Algorithm

- ❑ Philosopher grab a chopstick by calling wait() operation on that semaphore
- ❑ Release the chopstick by calling signal()
- ❑ Each chopstick — binary semaphore with initial value 1
- ❑ The structure of Philosopher *i*:

```
do {  
    wait (chopstick[i] );  
    wait (chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

- ❑ What is the problem with this algorithm?





Dining-Philosophers Problem Algorithm

- ❓ What is the problem with this algorithm?
- ❓ Deadlock — when all philosophers are hungry at the same time
- ❓ Solution:
 - ❓ Allow almost 4 philosophers to sit in the table
 - ❓ Allow philosophers to pick up the chopsticks only if both chopsticks are available
 - ❓ Asymmetric solution: odd philosopher picks up left chopsticks first and even philosopher picks up right chopsticks first





Classical Problems of Synchronization

- ❑ Classical problems used to test newly-proposed synchronization schemes
 - ❑ Bounded-Buffer Problem
 - ❑ Readers and Writers Problem
 - ❑ Dining-Philosophers Problem



End of Chapter 7

